Autor: Miquel Cerdà Cifre

Tasca: BD06

Link: https://github.com/mcc-fp/DAW-BD-MiquelCerdaCifre/tree/9b15928c7e87c6e580cd1bc9688e7ec30e39f4fb/BD06

Loom: https://www.loom.com/share/9f5cd59146c44f60a8e2927e3b0b2130

Exercise 1

An employee cannot be inserted into the employees table if another employee with the title "Sales Manager" already exists in the same office. In other words, there can only be one "Sales Manager" per office (officeCode).

If a new employee with the title "Sales Manager" is inserted into an office that already has one, the trigger must throw an error with a message indicating that a "Sales Manager" already exists in that office. The trigger only prevents insertion and does not handle updates.

```
use classicmodels

DELIMITER $$

CREATE TRIGGER before_insert_sales_manager
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN

-- Using LIKE because in the dataset jobTitle values are:
-- 'Sales Manager (NA)', 'Sales Manager (EMEA)', etc.
-- There is no exact 'Sales Manager' value, then the trigger will never been activate it

   IF NEW.jobTitle LIKE 'Sales Manager%' THEN

      -- Check if a Sales Manager already exists in the same office
      IF EXISTS (
         SELECT 1
         FROM employees
         WHERE officeCode = NEW.officeCode
         AND jobTitle LIKE 'Sales Manager%'
      ) THEN

         SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'A Sales Manager already exists in this office';

      END IF;

   END IF;
END$$

DELIMITER ;
```

Testing trigger:

```
-- Insert that should FAIL (duplicate Sales Manager in same office)
INSERT INTO employees (
   employeeNumber,
   lastName,
   firstName,
   extension,
   email,
   officeCode,
   reportsTo,
   jobTitle
) VALUES (
   9999,
   'Test',
   'Manager',
   'x9999',
   'test@gmail.com',
   '1',
   NULL,
   'Sales Manager'
);

-- Insert that should SUCCEED (no Sales Manager in that office)
INSERT INTO employees (
   employeeNumber,
   lastName,
   firstName,
   extension,
   email,
   officeCode,
   reportsTo,
   jobTitle
) VALUES (
   9998,
   'New',
   'Manager',
   'x9998',
   'newtest@gmail.com',
   '2',
   NULL,
   'Sales Manager'
);
```

Action Output

	Time	Action	Response
1	19:54:15	CREATE TRIGGER before_insert_sales_manager BEFORE INSERT ON employees FOR EACH ROW...	0 row(s) affected
2	19:54:29	INSERT INTO employees (employeeNumber, lastName, firstName, extension, email,...	Error Code: 1644. A Sales Manager already exists in this office
3	19:54:41	INSERT INTO employees (employeeNumber, lastName, firstName, extension, email,...	1 row(s) affected

Exercise 2

A customer cannot have more than 3 active orders (statuses 'In Process', 'On Hold', or 'Shipped') at any given time. Create a trigger that checks this condition before inserting into the orders table, throwing an error with a custom message if the customer attempts to create a fourth active order. Demonstrate how to test the trigger.

```
USE classicmodels;

DELIMITER $$

CREATE TRIGGER before_insert_active_orders
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
   DECLARE active_orders_count INT;

   IF NEW.status IN ('In Process', 'On Hold', 'Shipped') THEN
      SELECT COUNT(*)
      INTO active_orders_count
      FROM orders
      WHERE customerNumber = NEW.customerNumber
        AND status IN ('In Process', 'On Hold', 'Shipped');

      IF active_orders_count >= 3 THEN
         SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'This customer already has 3 active orders and cannot create a fourth one';
      END IF;
   END IF;
END$$

DELIMITER ;
```

Testing trigger:

-- First we want to view the total active orders by customer to be able to try the trigger

```
SELECT customerNumber, COUNT(*) AS active_orders
FROM orders
WHERE status IN ('In Process', 'On Hold', 'Shipped')
GROUP BY customerNumber
ORDER BY active_orders DESC, customerNumber;
```

-- Insert which will give us an error because customerNumber 103 has already 3 orders active

```
INSERT INTO orders (
   orderNumber,
   orderDate,
   requiredDate,
```

```
    shippedDate,
    status,
    comments,
    customerNumber
) VALUES (
    99910,
    CURDATE(),
    DATE_ADD(CURDATE(), INTERVAL 7 DAY),
    NULL,
    'In Process',
    'Trigger test - should fail',
    103
);
```

-- Insert which will work because customerNumber 171 only has already 2 orders active

```
INSERT INTO orders (
    orderNumber,
    orderDate,
    requiredDate,
    shippedDate,
    status,
    comments,
    customerNumber
) VALUES (
    99911,
    CURDATE(),
    DATE_ADD(CURDATE(), INTERVAL 7 DAY),
    NULL,
    'In Process',
    'Trigger test - should work',
    171
);
```

Action Output

	Time	Action	Response
1	20:46:43	USE classicmodels	0 row(s) affected
2	20:46:43	CREATE TRIGGER before_insert_active_orders BEFORE INSERT ON orders FOR EACH ROW BEGIN...	0 row(s) affected
3	20:47:08	SELECT customerNumber, COUNT(*) AS active_orders FROM orders WHERE status IN ('In Proces...	98 row(s) returned
4	20:47:46	INSERT INTO orders (orderNumber, orderDate, requiredDate, shippedDate, status,...	Error Code: 1644. This customer already has 3 active orders and c...
5	20:48:00	INSERT INTO orders (orderNumber, orderDate, requiredDate, shippedDate, status,...	1 row(s) affected

Exercise 3

We want to ensure that every order placed has stock available in the inventory.
Before inserting an order detail into the orderdetails table, verify that the product is in stock and that the requested quantity does not exceed the available stock (quantityInStock).
If the product is not in stock or the requested quantity exceeds the stock, the trigger must throw an error with a descriptive message (different for each case). Demonstrate how to test the trigger.

```
USE classicmodels;

DELIMITER $$

CREATE TRIGGER before_insert_orderdetails_check_stock
BEFORE INSERT ON orderdetails
FOR EACH ROW
BEGIN
   DECLARE v_stock INT DEFAULT NULL;

   SELECT quantityInStock
   INTO v_stock
   FROM products
   WHERE productCode = NEW.productCode;

   IF v_stock IS NULL THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'The product does not exist';
   END IF;

   IF v_stock = 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'This product is out of stock';
   END IF;

   IF NEW.quantityOrdered > v_stock THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Requested quantity exceeds available stock';
   END IF;
END$$

DELIMITER ;
```

Testing trigger:

```
SELECT productCode, productName, quantityInStock
FROM products
ORDER BY quantityInStock ASC, productCode;

-- Since S24_2000 has stock 15, requesting 10 units should succeed

INSERT INTO orderdetails (
   orderNumber,
   productCode,
   quantityOrdered,
   priceEach,
   orderLineNumber
) VALUES (
   10100,
   'S24_2000',
   10,
   76.17,
   99
);

-- The same product has stock 15, so requesting 20 units should fail

INSERT INTO orderdetails (
   orderNumber,
   productCode,
   quantityOrdered,
   priceEach,
   orderLineNumber
) VALUES (
   10101,
   'S24_2000',
   20,
   76.17,
   1
);

/* To test the out-of-stock case, since no product has stock = 0,
  we temporarily set one product's stock to 0 */

UPDATE products
SET quantityInStock = 0
WHERE productCode = 'S24_2000';
```

-- Now the product is out of stock, so any request should fail

```
INSERT INTO orderdetails (
   orderNumber,
   productCode,
   quantityOrdered,
   priceEach,
   orderLineNumber
) VALUES (
   10102,
   'S24_2000',
   1,
   76.17,
   1
);
```

Action Output

	Time	Action	Response
1	21:06:43	USE classicmodels	0 row(s) affected
2	21:06:43	DROP TRIGGER before_insert_orderdetails_check_stock	0 row(s) affected
3	21:06:43	CREATE TRIGGER before_insert_orderdetails_check_stock BEFORE INSERT ON orderdetails FOR...	0 row(s) affected
4	21:06:51	SELECT productCode, productName, quantityInStock FROM products ORDER BY quantityInStock...	110 row(s) returned
5	21:07:18	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach,...	1 row(s) affected
6	21:07:27	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach,...	Error Code: 1644. Requested quantity exceeds available stock
7	21:07:32	UPDATE products SET quantityInStock = 0 WHERE productCode = 'S24_2000'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
8	21:07:39	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach,...	Error Code: 1644. This product is out of stock

Exercise 4

Create a procedure named delete_employee to delete an employee from the employees table in classicmodels. Before deletion, ensure the following conditions are met:

1. Customers assigned to the employee must be reassigned to the employee's supervisor.
2. Employees supervised by the employee must be reassigned to the employee's supervisor.
3. If the employee has no supervisor (i.e., they are the president), they cannot be deleted, and a custom error must be returned.

The procedure must accept the employeeNumber as a parameter and return a custom error if the employeeNumber does not exist.

```
USE classicmodels;

DELIMITER $$

CREATE PROCEDURE delete_employee(IN p_employeeNumber INT)
BEGIN
  DECLARE v_supervisor INT DEFAULT NULL;
  DECLARE v_exists INT DEFAULT 0;

  SELECT COUNT(*)
  INTO v_exists
  FROM employees
  WHERE employeeNumber = p_employeeNumber;

  IF v_exists = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'The employeeNumber does not exist';
  END IF;

  SELECT reportsTo
  INTO v_supervisor
  FROM employees
  WHERE employeeNumber = p_employeeNumber;

  IF v_supervisor IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'The president cannot be deleted because they have no supervisor';
  END IF;

  UPDATE customers
  SET salesRepEmployeeNumber = v_supervisor
  WHERE salesRepEmployeeNumber = p_employeeNumber;

  UPDATE employees
  SET reportsTo = v_supervisor
  WHERE reportsTo = p_employeeNumber;

  DELETE FROM employees
```

```
    WHERE employeeNumber = p_employeeNumber;
END$$

DELIMITER ;
```

Testing the procedure

```
-- Valid case
CALL delete_employee(1165);

-- Invalid case: employee does not exist
CALL delete_employee(999999);

-- Invalid case: president cannot be deleted
CALL delete_employee(1002);
```

Action Output

	Time	Action	Response
1	21:18:26	USE classicmodels	0 row(s) affected
2	21:18:26	CREATE PROCEDURE delete_employee(IN p_employeeNumber INT) BEGIN DECLARE v_supervis...	0 row(s) affected
3	21:18:26	-- Valid case CALL delete_employee(1165)	1 row(s) affected
4	21:18:26	CALL delete_employee(999999)	Error Code: 1644. The employeeNumber does not exist
5	21:18:50	CALL delete_employee(1002)	Error Code: 1644. The president cannot be deleted because they have no superviso

Exercise 5

Create a procedure named customers_sales_report that accepts three parameters:

1. The office name (office_name) as an input parameter.
2. The year (year_param) as an input parameter.
3. An output parameter that returns a detailed report of monthly sales for the employees of that office during the specified year (based on orderDate).

The report must include:

- The full name of the employee.
- The customer's name (customerName).
- The total sales made by the employee for that customer during the year.

If no sales are found, the procedure must return a message indicating that no data is available

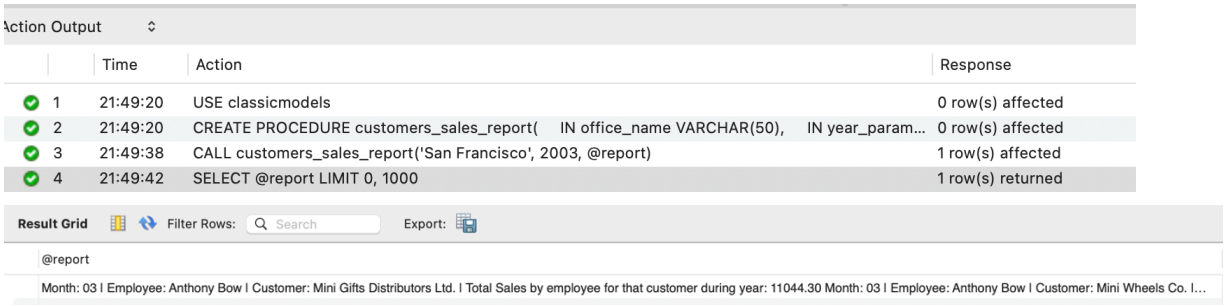
```
USE classicmodels;

DELIMITER $$

CREATE PROCEDURE customers_sales_report(
   IN office_name VARCHAR(50),
   IN year_param INT,
   OUT report_out LONGTEXT
)
BEGIN
   SET SESSION group_concat_max_len = 1000000;

   SELECT GROUP_CONCAT(
            CONCAT(
               'Month: ', LPAD(report_data.month_num, 2, '0'),
               ' | Employee: ', report_data.employee_name,
               ' | Customer: ', report_data.customer_name,
               ' | Total Sales by employee for that customer during year: ',
               ROUND(report_data.total_sales, 2)
            )
            ORDER BY report_data.month_num, report_data.employee_name, report_data.customer_name
            SEPARATOR '\n'
         )
   INTO report_out
   FROM (
      SELECT
         MONTH(o.orderDate) AS month_num,
         CONCAT(e.firstName, ' ', e.lastName) AS employee_name,
         c.customerName AS customer_name,
         SUM(od.quantityOrdered * od.priceEach) AS total_sales
      FROM offices ofc
      JOIN employees e
         ON e.officeCode = ofc.officeCode
      JOIN customers c
         ON c.salesRepEmployeeNumber = e.employeeNumber
      JOIN orders o
         ON o.customerNumber = c.customerNumber
```

```
    JOIN orderdetails od
      ON od.orderNumber = o.orderNumber
    WHERE ofc.city = office_name
     AND YEAR(o.orderDate) = year_param
    GROUP BY
      MONTH(o.orderDate),
      e.employeeNumber,
      e.firstName,
      e.lastName,
      c.customerNumber,
      c.customerName
  ) AS report_data;

  IF report_out IS NULL THEN
    SET report_out = 'No data available for the selected office and year';
  END IF;
END$$

DELIMITER ;
```

Testing the procedure

-- For example, one office city in the dataset is San Francisco.

```
CALL customers_sales_report('San Francisco', 2003, @report);
SELECT @report;
```



Action Output

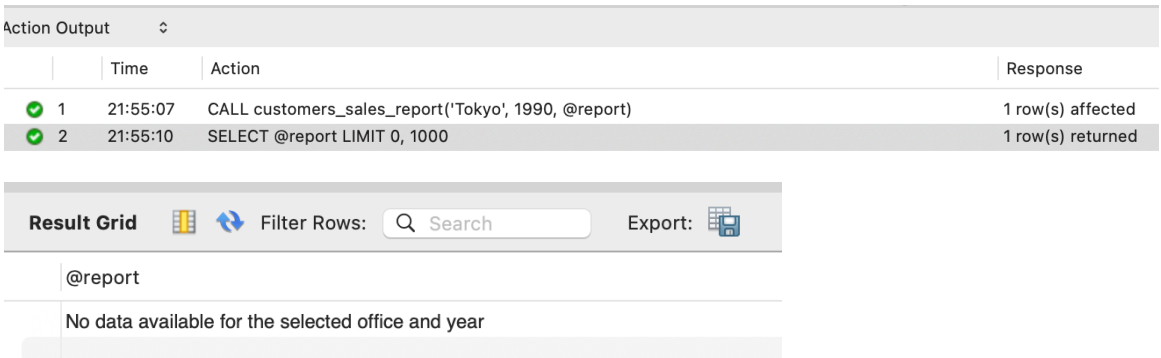
	Time	Action	Response
1	21:49:20	USE classicmodels	0 row(s) affected
2	21:49:20	CREATE PROCEDURE customers_sales_report(IN office_name VARCHAR(50), IN year_param...	0 row(s) affected
3	21:49:38	CALL customers_sales_report('San Francisco', 2003, @report)	1 row(s) affected
4	21:49:42	SELECT @report LIMIT 0, 1000	1 row(s) returned

Result Grid Filter Rows: Search Export:

@report
Month: 03 I Employee: Anthony Bow I Customer: Mini Gifts Distributors Ltd. I Total Sales by employee for that customer during year: 11044.30 Month: 03 I Employee: Anthony Bow I Customer: Mini Wheels Co. I...

-- Test with no data

```
CALL customers_sales_report('Tokyo', 1990, @report);
SELECT @report;
```



Action Output

	Time	Action	Response
1	21:55:07	CALL customers_sales_report('Tokyo', 1990, @report)	1 row(s) affected
2	21:55:10	SELECT @report LIMIT 0, 1000	1 row(s) returned

Result Grid Filter Rows: Search Export:

@report
No data available for the selected office and year

Exercise 6

Create a function that takes a customerNumber (customer ID) as a parameter and returns a string summarizing the number of orders and products purchased by that customer

```
USE classicmodels;

DELIMITER $$

CREATE FUNCTION customer_summary(p_customerNumber INT)
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
   DECLARE v_orders INT DEFAULT 0;
   DECLARE v_products INT DEFAULT 0;

   -- Count number of orders
   SELECT COUNT(*)
   INTO v_orders
   FROM orders
   WHERE customerNumber = p_customerNumber;

   -- Count number of products purchased
   SELECT COALESCE(SUM(od.quantityOrdered), 0)
   INTO v_products
   FROM orders o
   JOIN orderdetails od
      ON o.orderNumber = od.orderNumber
   WHERE o.customerNumber = p_customerNumber;

   -- Return formatted summary
   RETURN CONCAT(
      'Customer ', p_customerNumber,
      ' has ', v_orders, ' orders and ',
      v_products, ' products purchased'
   );
END$$

DELIMITER ;
```

Testing the function

```
-- Case with orders
SELECT customer_summary(103);
```

Action Output

	Time	Action	
1	22:10:16	USE classicmodels	0
2	22:10:16	CREATE FUNCTION customer_summary(p_customerNumber INT) RETURNS VARCHAR(255) DETE...	0
3	22:10:25	SELECT customer_summary(103) LIMIT 0, 1000	1

Result Grid Filter Rows: Search

customer_summary(103)
Customer 103 has 3 orders and 270 products purchased

-- Case without orders

SELECT customer_summary(999);

Action Output

	Time	Action
1	22:13:30	SELECT customer_summary(999) LIMIT 0, 1000

Result Grid Filter Rows: Search Export:

customer_summary(999)
Customer 999 has 0 orders and 0 products purchased